Document number:	P0651
Date:	2017-06-17
Project:	C++ Extensions for Ranges
Reply-to:	Eric Niebler <eniebler@boost.org>
Audience:	Library Working Group

Switch the Ranges TS to Use Variable Concepts

1 Synopsis

Currently, the Ranges TS uses function-style concepts. That’s because in a few places, concept names are “overloaded”. Rather than have some variable concepts and some function concepts, the Ranges TS opted for function concepts everywhere. The has proven unsatisfactory, because it fills function signatures with useless syntactic noise (e.g., `()`).

Also, committee in Kona (2016) expressed concern at there being multiple ways to define a concept. There seemed interest in eliminating function-style concept definitions. The Ranges TS was given as an example of where function-style concepts were used. But the Ranges TS does not need them and would in fact probably be better off without them.

The reasons for dropping function-style concepts from the Ranges TS then are:

1. To eliminate syntactic noise.
2. To eliminate user confusion that comes at a concept’s point-of-use, where sometimes parens are needed (in `requires` clauses) and sometimes not (when the concept is used as a placeholder).
3. To avoid depending on a feature that may get dropped.
4. To avoid becoming a reason to keep a little-loved feature.

There is really only one reason for keeping function-style concepts in the Ranges TS:

1. In several places, the Ranges TS defines concepts with similar semantic meaning but different numbers of parameters; function-style concepts neatly captures this intent by allowing these concepts to share a name.

2 Proposed Solution

We propose respecifying the Ranges TS in terms of variable-style concepts. There are three cases to handle:

1. Non-overloaded concepts
2. Cross-type concepts
3. Variable-argument concepts

2.1 Non-overloaded concepts

In the case of concepts that are not overloaded, changing to variable concepts is purely a syntactic rewrite. For example, the following function-style concept:

```
template <class T>
concept bool Movable() {
  return MoveConstructible<T>() &&
    Assignable<T&, T>() &&
    Swappable<T&>();
}
```

would become:

```
template <class T>
concept bool Movable =
  MoveConstructible<T> &&
  Assignable<T&, T> &&
  Swappable<T&>;
```

Obviously, all uses of this concept would also need to be changed to drop the trailing empty parens ("`()`"), if any.

2.2 Cross-type concepts

Some binary concepts offer a unary form to mean "same type", such that `Concept<A>()` is semantically identical to `Concept<A, A>()` (e.g., `EqualityComparable`). In these cases, a simple rewrite into a variable form will not result in valid code, since variable concepts cannot be overloaded. In these cases, we must find a different spelling for the unary and binary forms.

The suggestion is to use the suffix `With` for the binary form. So, `EqualityComparable<int>` would be roughly equivalent to `EqualityComparableWith<int, int>`. This follows the precedent set by the type traits `is_swappable` and `is_swappable_with`.

The concepts in the Ranges TS to which this applies are:

- `EqualityComparable`
- `Swappable`
- `StrictTotallyOrdered`

This pattern also appears in the relation concepts:

- `Relation`
- `StrictWeakOrder`

However, the forms `Relation<R, T>()` and `StrictWeakOrder<R, T>()` are used nowhere in the Ranges TS and can simply be dropped with no impact.

2.3 Variable-argument concepts

The concepts that have to do with callables naturally permit a variable number of arguments and are best expressed using variadic parameters packs. However, the *indirect* callable concepts used to constrain the higher-order STL algorithms are fixed-arity (not variadic) so as to be able to check callability with the cross-product of the iterators' associated types. The STL algorithms only ever deal with unary and binary callables, so the indirect callable concepts are "overloaded" on zero, one, or two arguments.

The affected concepts are:

- `IndirectInvocable`
- `IndirectRegularInvocable`
- `IndirectPredicate`

(The concepts `IndirectRelation` and `IndirectStrictWeakOrder` are unaffected because they are not overloaded.)

The concept `IndirectInvocable` is used to constrain the `for_each` algorithm, where the function

object it constrains is unary. So, we suggest dropping the nullary and binary forms of this concept and renaming `IndirectInvocable` to `IndirectUnaryInvocable`.

Likewise, the concept `IndirectRegularInvocable` is used to constrain the `projected` class template, where the function object it constrains is unary. So, we suggest dropping the nullary and binary forms of this concept and renaming `IndirectRegularInvocable` to `IndirectRegularUnaryInvocable`.

We observe that `IndirectPredicate` is only ever used to constrain unary predicates (once we fix ericniebler/stl2#411), so we suggest dropping the binary form and renaming `IndirectPredicate` to `IndirectUnaryPredicate`.

3 Discussion

Should the committee ever decide to permit variable-style concepts to be overloaded, we could decide to revert the name changes proposed in this document. For example, we could offer `EqualityComparable<A, B>` as an alternate syntax for `EqualityComparableWith<A, B>`, and deprecate `EqualityComparableWith`.

3.1 Alternative Solutions

The following solutions have been considered and dismissed.

3.1.1 Leave Function-Style Intact

There is nothing wrong *per se* with leaving the concepts as function-style. The Ranges TS is based on the Concepts TS as published, which supports the syntax. Giving up function-style concepts forces us to come up with unique names for semantically similar things, including the admittedly awful `IndirectUnaryInvocable` and friends.

This option comes with a few lasting costs, already discussed above. We feel the costs of sticking with function-style concepts outweigh the benefits of being able to "overload" concept names.

4 Implementation Experience

All the interface changes suggested in this document have been implemented and tested in the Ranges TS's reference implementation at https://github.com/CaseyCarter/cmcstl2. The change was straightforward and unsurprising.

5 Proposed Design

[*Editorial note:* In places where a purely mechanical transformation is sufficient, rather than show all the diffs (which would be overwhelming and tend to obsure the more meaningful edits), we describe the transformation, give an example, and instruct the editor to make the mechanical change everywhere applicable. In other cases, where concepts change name or need to be respecified, the changes are shown explicitly with diff marks.]

In all places in the document where concept checks are applied with a trailing set of empty parens ("`()`"), remove the parens.

5.1 Section "Concepts library" ([concepts.lib])

In "Header `<experimental/ranges/concepts>` synopsis" ([concepts.lib.synopsis]), except where noted below, change all the concept definitions from function-style concepts to variable-style concepts, following the pattern for `Same` shown below:

```
template <class T, class U>
concept bool Same~~() {~~ =
  ~~return~~ see below;
~~}~~
```

Change the second (binary) forms of `Swappable`, `EqualityComparable`, and `StrictTotallyOrdered` as

follows:

```
template <class T, class U>
concept bool SwappableWith() { =
  return see below;
}

template <class T, class U>
concept bool EqualityComparableWith() { =
  return see below;
}

template <class T, class U>
concept bool StrictTotallyOrderedWith() { =
  return see below;
}
```

In addition, remove the following two concept declarations:

```
template <class R, class T>
concept bool Relation() {
  return see below;
}

template <class R, class T>
concept bool StrictWeakOrder() {
  return see below;
}
```

Make the corresponding edits in sections 7.3 [concepts.lib.corelang] through section 7.6 [Callable concepts], except as follows.

In the section "Concept `Movable` ([concepts.lib.object.movable]), change the definition of `Movable` as follows (includes changes from P0547R1 and ericniebler/stl2#174 "Swappable concept and P0185 swappable traits"):

```
template <class T>
concept bool Movable() { =
  return std::is_object<T>::value && // see below
    MoveConstructible<T> &&
    Assignable<T&, T> &&
    Swappable<T&>;
}
```

In section "Concept `Swappable`" ([concepts.lib.corelang.swappable]), change the name of the binary form of `Swappable` to `SwappableWith` as follows:

[*Editorial note:* This includes the resolutions of ericniebler/stl2#155 "Comparison concepts and reference types" and ericniebler/stl2#174 "Swappable concept and P0185 swappable traits".]

```
template <class T>
concept bool Swappable() { =
  return requires(T&& a, T&& b) {
    ranges::swap(std::forward<T>(a), std::forward<T>(b));
  };
}

template <class T, class U>
concept bool SwappableWith =() {
  return Swappable<T>() &&
    Swappable<U>() &&
    CommonReference<
      const remove_reference_t<T>&,
      const remove_reference_t<U>&>() &&
    requires(T&& t, U&& u) {
      ranges::swap(std::forward<T>(t), std::forward<T>(t));
      ranges::swap(std::forward<U>(u), std::forward<U>(u));
      ranges::swap(std::forward<T>(t), std::forward<U>(u));
      ranges::swap(std::forward<U>(u), std::forward<T>(t));
    };
}
```

In [concepts.lib.corelang.swappable]/p2, change the two occurrences of `Swappable<T, U>()` to `SwappableWith<T, U>`.

Change section “Concept `EqualityComparable`” ([concepts.lib.compare.equalitycomparable])/p3-4 as follows:

> 3 [*Note:* The requirement that the expression `a == b` is equality preserving implies that `==` is reflexive, transitive, and symmetric. -- *end note*]

```
template <class T, class U>
concept bool EqualityComparableWith~~() {~~ =
  ~~return~~
    EqualityComparable<T>~~()~~ &&
    EqualityComparable<U>~~()~~ &&
    CommonReference<
      const remove_reference_t<T>&,
      const remove_reference_t<U>&>~~()~~ &&
    EqualityComparable<
      common_reference_t<
        const remove_reference_t<T>&,
        const remove_reference_t<U>&>>~~()~~ &&
    WeaklyEqualityComparable<T, U>~~()~~;
~~}~~
```

> 4 Let `a` be a `const` lvalue of type `remove_reference_t<T>`, `b` be a `const` lvalue of type `remove_reference_t<U>`, and `C` be `common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>`. Then `EqualityComparableWith<T, U>~~()~~` is satisfied if and only if:
>
> (4.1) -- `bool(t == u) == bool(C(t) == C(u))`.

[*Note:*This includes the resolution of ericniebler/stl2#155.]

In section “Concept `StrictTotallyOrdered`” ([concepts.lib.corelang.stricttotallyordered]), change the name of the binary form of `StrictTotallyOrdered` to `StrictTotallyOrderedWith` as follows:

[*Editorial note:* This includes the resolution of ericniebler/stl2#155 “Comparison concepts and reference types”.]

```
template <class T>
concept bool StrictTotallyOrderedWith~~() {~~ =
  ~~return~~
    StrictTotallyOrdered<T>~~()~~ &&
    StrictTotallyOrdered<U>~~()~~ &&
    CommonReference<
      const remove_reference_t<T>&,
      const remove_reference_t<U>&>~~()~~ &&
    StrictTotallyOrdered<
      common_reference_t<
        const remove_reference_t<T>&,
        const remove_reference_t<U>&>>~~()~~ &&
    EqualityComparableWith<T, U>~~()~~ &&
    requires(const remove_reference_t<T>& t,
             const remove_reference_t<U>& u) {
      { t < u } -> Boolean&&;
      { t > u } -> Boolean&&;
      { t <= u } -> Boolean&&;
      { t >= u } -> Boolean&&;
      { u < t } -> Boolean&&;
      { u > t } -> Boolean&&;
      { u <= t } -> Boolean&&;
      { u >= t } -> Boolean&&;
    };
~~}~~
```

In [concepts.lib.corelang.stricttotallyordered]/p2, change the occurrence of `StrictTotallyOrdered<T, U>()` to `StrictTotallyOrderedWith<T, U>`.

Change section “Concept `Relation`” ([concepts.lib.callable.relation]) as follows:

[*Editorial note:* This includes the resolution of ericniebler/stl2#155 “Comparison concepts and reference types”.]

```
~~template <class R, class T>~~
~~concept bool Relation() {~~
```

```
  return Predicate<R, T, T>;
}

template <class R, class T, class U>
concept bool Relation() {
  return RelationPredicate<R, T, T>() &&
    RelationPredicate<R, U, U>() &&
    CommonReference<
      const remove_reference_t<T>&,
      const remove_reference_t<U>&>() &&
    RelationPredicate<R,
      common_reference_t<
        const remove_reference_t<T>&,
        const remove_reference_t<U>&>,
      common_reference_t<
        const remove_reference_t<T>&,
        const remove_reference_t<U>&>>() &&
    Predicate<R, T, U>() &&
    Predicate<R, U, T>();
}
```

Change section “Concept `StrictWeakOrder`” ([concepts.lib.callable.strictweakorder]) as follows:

```
template <class R, class T>
concept bool StrictWeakOrder() {
  return Relation<R, T>();
}

template <class R, class T, class U>
concept bool StrictWeakOrder() {
  return Relation<R, T, U>();
}
```

5.2 Section “General utilities library” ([utilities])

Change section “Header `<experimental/ranges/utility>` synopsis” ([utility]/p2) as follows (includes the resolution of ericniebler/stl2#174 “Swappable concept and P0185 swappable traits”):

```
// 8.5.2, struct with named accessors
template <class T>
concept bool TagSpecifier() {
  return see below ;
}

template <class F>
concept bool TaggedType() {
  return see below ;
}

template <class Base, TagSpecifier... Tags>
  requires sizeof...(Tags) <= tuple_size<Base>::value
struct tagged :
  [...]
  tagged& operator=(U&& u) noexcept(see below);
  void swap(tagged& that) noexcept(see below)
    requires Swappable<Base&>;
  friend void swap(tagged&, tagged&) noexcept(see below)
    requires Swappable<Base&>;
};
```

Make the accompanying edit to the definitions of `TagSpecifier` and `TaggedType` in section “Class template `tagged`” [taggedtup.tagged]/p2, and to the detailed specifications of `tagged::swap` and the non-member `swap(tagged&, tagged&)` overload in [taggedtup.tagged]/p20 and p23

In the declarations of `equal_to<void>::operator()` and `not_equal_to<void>::operator()` in section “Comparisons” [comparisons]/p8-9, change `EqualityComparable<T, U>()` to `EqualityComparableWith<T, U>`.

In the declarations of `greater<void>::operator()`, `less<void>::operator()`, `greater_equal<void>::operator()`, and `less_equal<void>::operator()` in section “Comparisons” [comparisons]/p10-13, change `StrictTotallyOrdered<T, U>()` to `StrictTotallyOrderedWith<T, U>`.

5.3 Section “Iterators library” ([iterators])

In “Header `<experimental/ranges/iterators>` synopsis” ([concepts.lib.synopsis]), except where noted below, change all the function-style concept definitions to variable-style concepts.

In section “Header `<experimental/ranges/iterator>` synopsis” ([iterator.synopsis]), make the following changes:

```
// 9.4, indirect callable requirements:
// 9.4.2, indirect callables:
template <class F>
concept bool IndirectInvocable() {
  return see below ;
}
template <class F, class I>
concept bool IndirectUnaryInvocable() { =
  return see below ;
}
template <class F, class I1, class I2>
concept bool IndirectInvocable() {
  return see below ;
}
template <class F>
concept bool IndirectRegularInvocable() {
  return see below ;
}
template <class F, class I>
concept bool IndirectRegularUnaryInvocable() { =
  return see below ;
}
template <class F, class I1, class I2>
concept bool IndirectRegularInvocable() {
  return see below ;
}
template <class F, class I>
concept bool IndirectUnaryPredicate() { =
  return see below ;
}
template <class F, class I1, class I2>
concept bool IndirectPredicate() {
  return see below ;
}

[...]

// 9.4.3, projected:
template <Readable I, IndirectRegularUnaryInvocable<I> Proj>
struct projected;

[...]

// 9.7, predefined iterators and sentinels:
// 9.7.1, reverse iterators:
template <BidirectionalIterator I> class reverse_iterator;

template <class I1, class I2>
    requires EqualityComparableWith<I1, I2>()
  bool operator==(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);
template <class I1, class I2>
    requires EqualityComparableWith<I1, I2>()
  bool operator!=(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);
template <class I1, class I2>
    requires StrictTotallyOrderedWith<I1, I2>()
  bool operator<(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);
template <class I1, class I2>
requires StrictTotallyOrderedWith<I1, I2>()
  bool operator>(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);
template <class I1, class I2>
requires StrictTotallyOrderedWith<I1, I2>()
```

```
  bool operator>=(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);
template <class I1, class I2>
requires StrictTotallyOrderedWith<I1, I2>() 
  bool operator<=(
    const reverse_iterator<I1>& x,
    const reverse_iterator<I2>& y);

[...]

// 9.7.3, move iterators and sentinels:
template <InputIterator I> class move_iterator;
template <class I1, class I2>
    requires EqualityComparableWith<I1, I2>() 
  bool operator==(
    const move_iterator<I1>& x, const move_iterator<I2>& y);
template <class I1, class I2>
    requires EqualityComparableWith<I1, I2>() 
  bool operator!=(
    const move_iterator<I1>& x, const move_iterator<I2>& y);
template <class I1, class I2>
    requires StrictTotallyOrderedWith<I1, I2>() 
  bool operator<(
    const move_iterator<I1>& x, const move_iterator<I2>& y);
template <class I1, class I2>
    requires StrictTotallyOrderedWith<I1, I2>() 
  bool operator<=(
    const move_iterator<I1>& x, const move_iterator<I2>& y);
template <class I1, class I2>
    requires StrictTotallyOrderedWith<I1, I2>() 
  bool operator>(
    const move_iterator<I1>& x, const move_iterator<I2>& y);
template <class I1, class I2>
    requires StrictTotallyOrderedWith<I1, I2>() 
  bool operator>=(
    const move_iterator<I1>& x, const move_iterator<I2>& y);

[...]

template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
  bool operator==(
    const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
    requires EqualityComparableWith<I1, I2>() 
  bool operator==(
    const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
template <class I1, class I2, Sentinel<I2> S1, Sentinel<I1> S2>
  bool operator!=(
    const common_iterator<I1, S1>& x, const common_iterator<I2, S2>& y);
```

[*Editorial note:* The resolution of ericniebler/stl2#286 changes `indirect_result_of` to no longer use `IndirectInvocable`, so no change is necessary there. – *end note*]

Change section “Indirect callables” [indirectcallable.indirectinvocable] as follows:

```
template <class F>
concept bool IndirectInvocable() {
  return CopyConstructible<F>() &&
    Invocable<F&>();
}
template <class F, class I>
concept bool IndirectUnaryInvocable() { =
  return Readable<I>() &&
    CopyConstructible<F>() &&
    Invocable<F&, value_type_t<I>&>() &&
    Invocable<F&, reference_t<I>>() &&
    Invocable<F&, iter_common_reference_t<I>>();
}
template <class F, class I1, class I2>
concept bool IndirectInvocable() {
  return Readable<I1>() && Readable<I2>() &&
    CopyConstructible<F>() &&
    Invocable<F&, value_type_t<I1>&, value_type_t<I2>&>() &&
    Invocable<F&, value_type_t<I1>&, reference_t<I2>>() &&
    Invocable<F&, reference_t<I1>, value_type_t<I2>&>() &&
    Invocable<F&, reference_t<I1>, reference_t<I2>>() &&
    Invocable<F&, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
```

```
}

template <class F>
concept bool IndirectRegularInvocable() {
  return CopyConstructible<F>() &&
    RegularInvocable<F&>();
}
template <class F, class I>
concept bool IndirectRegularUnaryInvocable() =
  return Readable<I>() &&
    CopyConstructible<F>() &&
    RegularInvocable<F&, value_type_t<I>&>() &&
    RegularInvocable<F&, reference_t<I>>() &&
    RegularInvocable<F&, iter_common_reference_t<I>>();
}
template <class F, class I1, class I2>
concept bool IndirectRegularInvocable() {
  return Readable<I1>() && Readable<I2>() &&
    CopyConstructible<F>() &&
    RegularInvocable<F&, value_type_t<I1>&, value_type_t<I2>&>() &&
    RegularInvocable<F&, value_type_t<I1>&, reference_t<I2>>() &&
    RegularInvocable<F&, reference_t<I1>, value_type_t<I2>&>() &&
    RegularInvocable<F&, reference_t<I1>, reference_t<I2>>() &&
    RegularInvocable<F&, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}

template <class F, class I>
concept bool IndirectUnaryPredicate() =
  return Readable<I>() &&
    CopyConstructible<F>() &&
    Predicate<F&, value_type_t<I>&>() &&
    Predicate<F&, reference_t<I>>() &&
    Predicate<F&, iter_common_reference_t<I>>();
}
template <class F, class I1, class I2>
concept bool IndirectPredicate() {
  return Readable<I1>() && Readable<I2>() &&
    CopyConstructible<F>() &&
    Predicate<F&, value_type_t<I1>&, value_type_t<I2>&>() &&
    Predicate<F&, value_type_t<I1>&, reference_t<I2>>() &&
    Predicate<F&, reference_t<I1>, value_type_t<I2>&>() &&
    Predicate<F&, reference_t<I1>, reference_t<I2>>() &&
    Predicate<F&, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}
```

In section “Class template `projected`” ([projected]), change the occurrence of `IndirectRegularInvocable` to `IndirectRegularUnaryInvocable`.

In [commonalgoreq.general]/p2, change the note to read:

> [...] `equal_to<>` requires its arguments satisfy `EqualityComparableWith` (7.4.3), and `less<>` requires its arguments satisfy `StrictTotallyOrderedWith` (7.4.4). [...]

In section “Class template `reverse_iterator`” ([reverse.iterator]), in the synopsis and in definitions of the relational operators ([reverse.iter.op==] through [reverse.iter.op<=]), change `EqualityComparable<I1, I2>()` to `EqualityComparableWith<I1, I2>`, and change `StrictTotallyOrdered<I1, I2>()` to `StrictTotallyOrderedWith<I1, I2>` as shown above in the `<experimental/ranges/iterator>` synopsis ([iterator.synopsis]).

In section “Class template `move_iterator`” ([move.iterator]), in the synopsis (p2) and in definitions of the relational operators ([[move.iter.op.comp]]), change `EqualityComparable<I1, I2>()` to `EqualityComparableWith<I1, I2>`, and change `StrictTotallyOrdered<I1, I2>()` to `StrictTotallyOrderedWith<I1, I2>` as shown above in the `<experimental/ranges/iterator>` synopsis ([iterator.synopsis]).

In section “Class template `move_sentinel`” ([move.sentinel]), in the `move_if` example in para 2, change `IndirectPredicate` to `IndirectUnaryPredicate`.

In section “Class template `common_iterator`” ([common.iterator]), in the synopsis (p2) and in definitions of the relational operators ([[common.iter.op.comp]]), change `EqualityComparable<I1, I2>()` to `EqualityComparableWith<I1, I2>` as shown above in the `<experimental/ranges/iterator>` synopsis ([iterator.synopsis]).

In section “Range requirements” ([ranges.requirements]), change all function-style concept definitions

to variable-style concept definitions.

5.4 Section “Algorithms library” [algorithms]

In “Header `<experimental/ranges/algorithm>` synopsis” ([algorithms.general]/p2), make the following changes:

```
template <InputIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  bool all_of(I first, S last, Pred pred, Proj proj = Proj{});
template <InputRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  bool all_of(Rng&& rng, Pred pred, Proj proj = Proj{});

template <InputIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  bool any_of(I first, S last, Pred pred, Proj proj = Proj{});
template <InputRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  bool any_of(Rng&& rng, Pred pred, Proj proj = Proj{});

template <InputIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  bool none_of(I first, S last, Pred pred, Proj proj = Proj{});
template <InputRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  bool none_of(Rng&& rng, Pred pred, Proj proj = Proj{});

template <InputIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryInvocable<projected<I, Proj>> Fun>
  tagged_pair<tag::in(I), tag::fun(Fun)>
    for_each(I first, S last, Fun f, Proj proj = Proj{});
template <InputRange Rng, class Proj = identity,
    IndirectUnaryInvocable<projected<iterator_t<Rng>, Proj>> Fun>
  tagged_pair<tag::in(safe_iterator_t<Rng>), tag::fun(Fun)>
    for_each(Rng&& rng, Fun f, Proj proj = Proj{});

[...]

template <InputIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  I find_if(I first, S last, Pred pred, Proj proj = Proj{});
template <InputRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  safe_iterator_t<Rng>
    find_if(Rng&& rng, Pred pred, Proj proj = Proj{});

template <InputIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  I find_if_not(I first, S last, Pred pred, Proj proj = Proj{});
template <InputRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  safe_iterator_t<Rng>
    find_if_not(Rng&& rng, Pred pred, Proj proj = Proj{});

[...]

template <InputIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  difference_type_t<I>
    count_if(I first, S last, Pred pred, Proj proj = Proj{});
template <InputRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  difference_type_t<iterator_t<Rng>>
    count_if(Rng&& rng, Pred pred, Proj proj = Proj{});

[...]

template <InputIterator I, Sentinel<I> S, WeaklyIncrementable O, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  requires IndirectlyCopyable<I, O>~~()~~
  tagged_pair<tag::in(I), tag::out(O)>
    copy_if(I first, S last, O result, Pred pred, Proj proj = Proj{});
template <InputRange Rng, WeaklyIncrementable O, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  requires IndirectlyCopyable<iterator_t<Rng>, O>~~()~~
  tagged_pair<tag::in(safe_iterator_t<Rng>), tag::out(O)>
```

```
    copy_if(Rng&& rng, O result, Pred pred, Proj proj = Proj{});

[...]

template <ForwardIterator I, Sentinel<I> S, class T, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  requires Writable<I, const T&>()
  I
    replace_if(I first, S last, Pred pred, const T& new_value, Proj proj = Proj{});
template <ForwardRange Rng, class T, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  requires Writable<iterator_t<Rng>, const T&>()
  safe_iterator_t<Rng>
    replace_if(Rng&& rng, Pred pred, const T& new_value, Proj proj = Proj{});

[...]

template <InputIterator I, Sentinel<I> S, class T, OutputIterator<const T&> O,
    class Proj = identity, IndirectUnaryPredicate<projected<I, Proj>> Pred>
  requires IndirectlyCopyable<I, O>()
  tagged_pair<tag::in(I), tag::out(O)>
    replace_copy_if(I first, S last, O result, Pred pred, const T& new_value,
                    Proj proj = Proj{});
template <InputRange Rng, class T, OutputIterator<const T&> O, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  requires IndirectlyCopyable<iterator_t<Rng>, O>()
  tagged_pair<tag::in(safe_iterator_t<Rng>), tag::out(O)>
    replace_copy_if(Rng&& rng, O result, Pred pred, const T& new_value,
                    Proj proj = Proj{});

[...]

template <ForwardIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  requires Permutable<I>()
  I remove_if(I first, S last, Pred pred, Proj proj = Proj{});
template <ForwardRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  requires Permutable<iterator_t<Rng>>()
  safe_iterator_t<Rng>
    remove_if(Rng&& rng, Pred pred, Proj proj = Proj{});

[...]

template <InputIterator I, Sentinel<I> S, WeaklyIncrementable O,
    class Proj = identity, IndirectUnaryPredicate<projected<I, Proj>> Pred>
  requires IndirectlyCopyable<I, O>()
  tagged_pair<tag::in(I), tag::out(O)>
    remove_copy_if(I first, S last, O result, Pred pred, Proj proj = Proj{});
template <InputRange Rng, WeaklyIncrementable O, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  requires IndirectlyCopyable<iterator_t<Rng>, O>()
  tagged_pair<tag::in(safe_iterator_t<Rng>), tag::out(O)>
    remove_copy_if(Rng&& rng, O result, Pred pred, Proj proj = Proj{});

[...]

template <InputIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  bool is_partitioned(I first, S last, Pred pred, Proj proj = Proj{});
template <InputRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  bool
  is_partitioned(Rng&& rng, Pred pred, Proj proj = Proj{});

template <ForwardIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  requires Permutable<I>()
  I partition(I first, S last, Pred pred, Proj proj = Proj{});
template <ForwardRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  requires Permutable<iterator_t<Rng>>()
  safe_iterator_t<Rng>
    partition(Rng&& rng, Pred pred, Proj proj = Proj{});

template <BidirectionalIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  requires Permutable<I>()
  I stable_partition(I first, S last, Pred pred, Proj proj = Proj{});
```

```
template <BidirectionalRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  requires Permutable<iterator_t<Rng>>() 
  safe_iterator_t<Rng>
    stable_partition(Rng&& rng, Pred pred, Proj proj = Proj{});

template <InputIterator I, Sentinel<I> S, WeaklyIncrementable O1, WeaklyIncrementable O2,
    class Proj = identity, IndirectUnaryPredicate<projected<I, Proj>> Pred>
  requires IndirectlyCopyable<I, O1>() && IndirectlyCopyable<I, O2>()
  tagged_tuple<tag::in(I), tag::out1(O1), tag::out2(O2)>
    partition_copy(I first, S last, O1 out_true, O2 out_false, Pred pred,
                   Proj proj = Proj{});
template <InputRange Rng, WeaklyIncrementable O1, WeaklyIncrementable O2,
    class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  requires IndirectlyCopyable<iterator_t<Rng>, O1>() &&
    IndirectlyCopyable<iterator_t<Rng>, O2>()
  tagged_tuple<tag::in(safe_iterator_t<Rng>), tag::out1(O1), tag::out2(O2)>
    partition_copy(Rng&& rng, O1 out_true, O2 out_false, Pred pred, Proj proj = Proj{});

template <ForwardIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  I partition_point(I first, S last, Pred pred, Proj proj = Proj{});
template <ForwardRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  safe_iterator_t<Rng>
  partition_point(Rng&& rng, Pred pred, Proj proj = Proj{});
```

In section “All of” ([alg.all_of]), change the signature of the `all_of` algorithm to match those shown in `<experimental/ranges/algorithm>` synopsis ([algorithms.general]/p2) above.

Likewise, do the same for the following algorithms:

- `any_of` in section “Any of” ([alg.any_of])
- `none_of` in section “None of” ([alg.none_of])
- `for_each` in section “For each” ([alg.for_each])
- `find_if` in section “Find” ([alg.find])
- `find_if_not` in section “Find” ([alg.find])
- `count_if` in section “Count” ([alg.count])
- `copy_if` in section “Copy” ([alg.copy])
- `replace_if` in section “Replace” ([alg.replace])
- `replace_copy_if` in section “Replace” ([alg.replace])
- `remove_if` in section “Remove” ([alg.remove])
- `remove_copy_if` in section “Remove” ([alg.remove])
- `is_partitioned` in section “Partitions” ([alg.partitions])
- `partition` in section “Partitions” ([alg.partitions])
- `stable_partition` in section “Partitions” ([alg.partitions])
- `partition_copy` in section “Partitions” ([alg.partitions])
- `partition_point` in section “Partitions” ([alg.partitions])

6 Acknowledgements

I would like to thank Casey Carter for his review feedback.